

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics Practical

Practical – V

Roll No.: T010	Name: Sagar Kandari
Class: TYIT	Batch: 01
Date of Assignment: 21/08/2026	Date/Time of Submission: 21/08/2026

5. Probability and Probability Distributions:

a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

```

a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

(1) ✓ 3.7s Python

np.random.seed(42)
print("----- DICE ROLL EXPERIMENT -----")

(2) ✓ 0.0s Python

----- DICE ROLL EXPERIMENT -----

# 1. Simulate 1,000 dice rolls
rolls = np.random.randint(1, 7, 1000)

print("\nFirst 20 Dice Outcomes:")
print(rolls[:20])

(4) ✓ 0.0s Python

First 20 Dice Outcomes:
[4 5 3 5 5 2 3 3 3 5 4 3 6 5 2 4 6 6 2 4]

# 2. Calculate experimental probability
frequency = pd.Series(rolls).value_counts().sort_index()
experimental_probability = frequency / 1000
print("\nFrequency of Each Outcome:")
print(frequency)
print("\nExperimental Probability:")
for number in range(1, 7):
    print(f"P({number}) = {experimental_probability[number]:.4f}")

(5) ✓ 0.0s Python

Frequency of Each Outcome:
1    181
2    164
3    154
4    174
5    172
6    155
Name: count, dtype: int64

Experimental Probability:
P(1) = 0.1810
P(2) = 0.1640
P(3) = 0.1540
P(4) = 0.1740
P(5) = 0.1720
P(6) = 0.1550

# 3. Calculate theoretical probability
theoretical_probability = 1 / 6
print("\nTheoretical Probability:")

```

```

for number in range(1, 7):
    print(f"P({number}) = {theoretical_probability:.4f}")

```

✓ 0.0s Python

...

Theoretical Probability:

P(1) = 0.1667
P(2) = 0.1667
P(3) = 0.1667
P(4) = 0.1667
P(5) = 0.1667
P(6) = 0.1667

```

# 4. Probability of getting 1, 2, 3, 4, 5, 6
print("\nProbability of Individual Outcomes:")
for number in range(1, 7):
    print(f"Number {number}: {experimental_probability[number]:.4f}")

```

✓ 0.0s Python

...

Probability of Individual Outcomes:

Number 1: 0.1810
Number 2: 0.1640
Number 3: 0.1540
Number 4: 0.1740
Number 5: 0.1720
Number 6: 0.1550

```

# 5. Probability of getting an even number
even_probability = np.mean(np.isin(rolls, [2, 4, 6]))
print("\nProbability of Even Number:")
print(f"Experimental Probability = {even_probability:.4f}")
print(f"Theoretical Probability = {3/6:.4f}")

```

✓ 0.0s Python

...

Probability of Even Number:

Experimental Probability = 0.4930
Theoretical Probability = 0.5000

```

# 6. Probability of getting an odd number
odd_probability = np.mean(np.isin(rolls, [1, 3, 5]))
print("\nProbability of Odd Number:")
print(f"Experimental Probability = {odd_probability:.4f}")
print(f"Theoretical Probability = {3/6:.4f}")

```

✓ 0.0s Python

...

Probability of Odd Number:

Experimental Probability = 0.5070
Theoretical Probability = 0.5000

```

# 7. Probability of getting a number greater than 4
greater_4 = np.mean(rolls > 4)
print("\nProbability of Number Greater Than 4:")
print(f"Experimental Probability = {greater_4:.4f}")
print(f"Theoretical Probability = {2/6:.4f}")

```

✓ 0.0s Python

...

Probability of Number Greater Than 4:

Experimental Probability = 0.3270
Theoretical Probability = 0.3333

```

# 8. Probability of getting a number less than 4
less_4 = np.mean(rolls < 4)
print("\nProbability of Number Less Than 4:")
print(f"Experimental Probability = {less_4:.4f}")
print(f"Theoretical Probability = {3/6:.4f}")

```

✓ 0.0s Python

...

Probability of Number Less Than 4:

Experimental Probability = 0.4990
Theoretical Probability = 0.5000

```
# 9. Probability of getting a number greater than or equal to 4
greater_equal_4 = np.mean(rolls >= 4)
print("\nProbability of Number Greater Than or Equal To 4:")
print(f"Experimental Probability = {greater_equal_4:.4f}")
print(f"Theoretical Probability = {3/6:.4f}")
```

[12] ✓ 0.0s

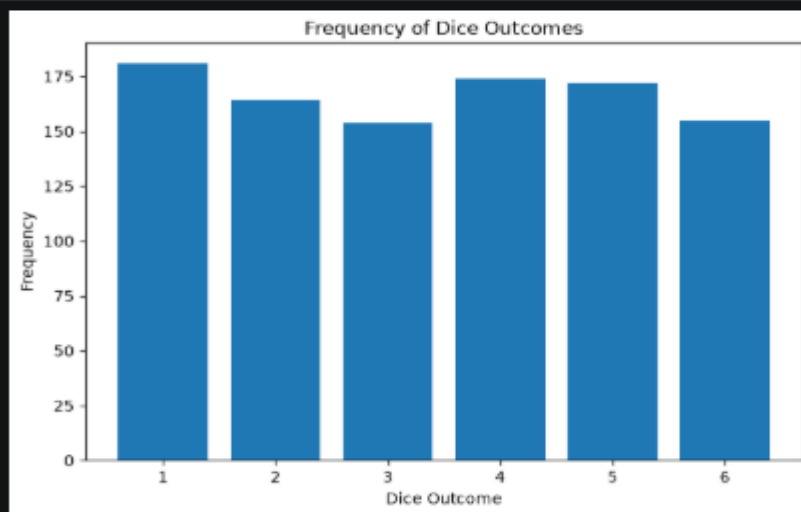
Python

```
Probability of Number Greater Than or Equal To 4:
Experimental Probability = 0.5010
Theoretical Probability = 0.5000
```

```
# 10. Bar chart showing frequency of outcomes 1 to 6
plt.figure(figsize=(8, 5))
plt.bar(frequency.index, frequency.values)
plt.xlabel("Dice Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Dice Outcomes")
plt.xticks(range(1, 7))
plt.show()
```

[13] ✓ 0.1s

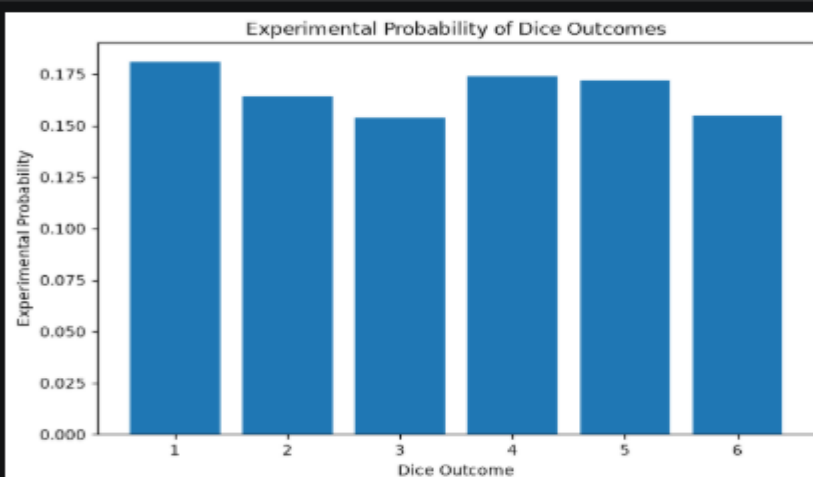
Python



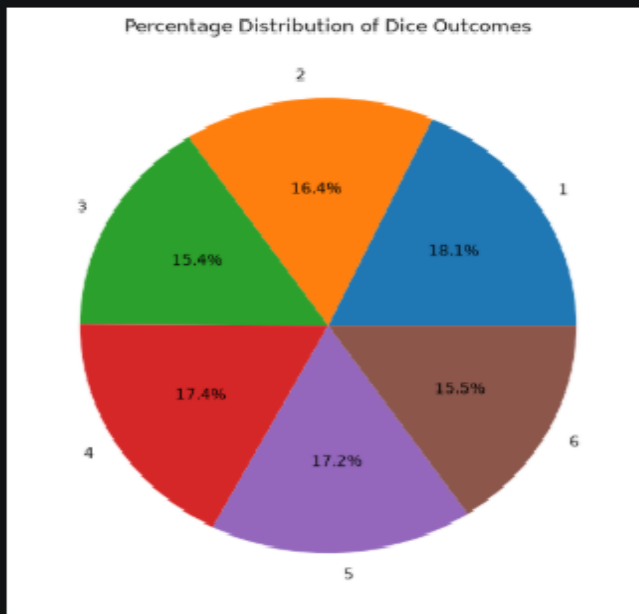
```
# 11. Bar chart showing experimental probability
plt.figure(figsize=(8, 5))
plt.bar(
    experimental_probability.index,
    experimental_probability.values
)
plt.xlabel("Dice Outcome")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Dice Outcomes")
plt.xticks(range(1, 7))
plt.show()
```

[14] ✓ 0.0s

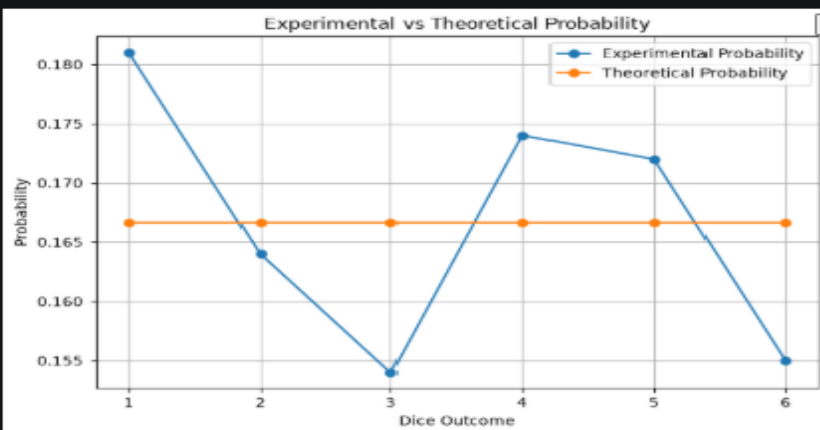
Python



```
# 12. Pie chart showing percentage distribution
plt.figure(figsize=(7, 7))
plt.pie(
    frequency.values,
    labels=frequency.index,
    autopct="%1.1f%%"
)
plt.title("Percentage Distribution of Dice Outcomes")
plt.show()
```

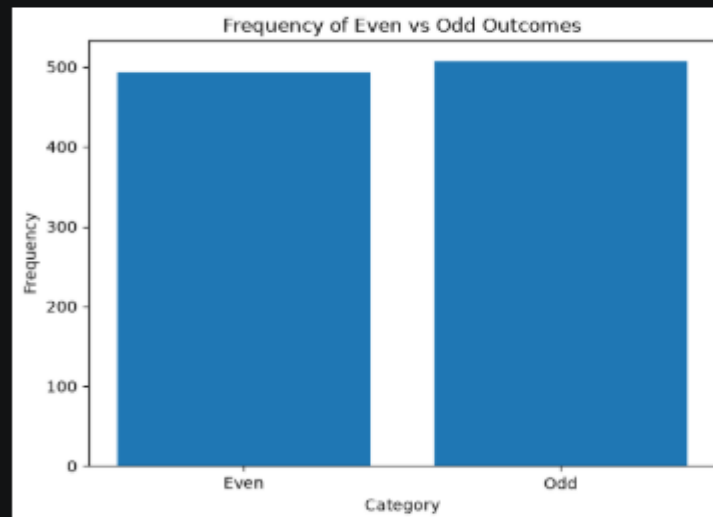


```
# 13. Line chart comparing experimental and theoretical probability
plt.figure(figsize=(8, 5))
plt.plot(
    range(1, 7),
    experimental_probability.values,
    marker="o",
    label="Experimental Probability"
)
plt.plot(
    range(1, 7),
    [theoretical_probability] * 6,
    marker="o",
    label="Theoretical Probability"
)
plt.xlabel("Dice Outcome")
plt.ylabel("Probability")
plt.title("Experimental vs Theoretical Probability")
plt.xticks(range(1, 7))
plt.legend()
plt.grid()
plt.show()
```



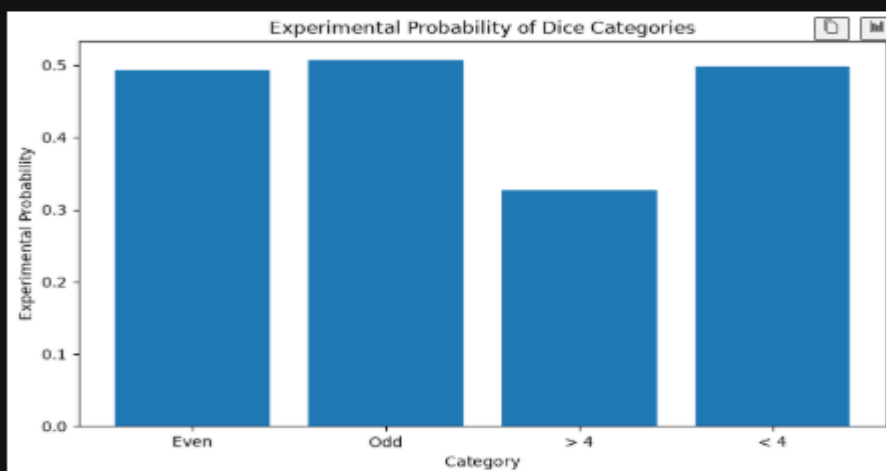
```
# 14. Chart showing frequency of Even vs Odd outcomes
even_count = np.sum(np.isin(rolls, [2, 4, 6]))
odd_count = np.sum(np.isin(rolls, [1, 3, 5]))
plt.figure(figsize=(7, 5))
plt.bar(
    ["Even", "Odd"],
    [even_count, odd_count]
)
plt.xlabel("Category")
plt.ylabel("Frequency")
plt.title("Frequency of Even vs Odd Outcomes")
plt.show()
```

[17]



```
# 15. Experimental probability of Even, Odd, Greater than 4, Less than 4
category_probabilities = {
    "Even": even_probability,
    "Odd": odd_probability,
    "> 4": greater_4,
    "< 4": less_4
}
plt.figure(figsize=(9, 5))
plt.bar(
    category_probabilities.keys(),
    category_probabilities.values()
)
plt.xlabel("Category")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Dice Categories")
plt.show()
print("\n----- DICE EXPERIMENT COMPLETED -----")
```

[18]



b. Generate random numbers from various probability distributions (normal,

uniform, exponential) and analyze their properties.

b. Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as pl
(19) ✓ 0.0s Python

np.random.seed(42)
print("----- PROBABILITY DISTRIBUTIONS -----")
(20) ✓ 0.0s Python

----- PROBABILITY DISTRIBUTIONS -----

# NORMAL DISTRIBUTION
normal_data = np.random.normal(
    loc=50,
    scale=10,
    size=1000
)
normal_series = pd.Series(normal_data)
(21) ✓ 0.0s Python

# 1. Generate 1,000 random numbers from normal distribution
print("\n----- NORMAL DISTRIBUTION -----")
print("First 20 Normal Distribution Values:")
print(normal_data[:20])
(22) ✓ 0.0s Python

----- NORMAL DISTRIBUTION -----
First 20 Normal Distribution Values:
[54.96714153 48.61735699 56.47688538 65.23029856 47.65846625 47.65863043
 65.79212816 57.67434729 45.30525614 55.42560044 45.36582307 45.34270246
 52.41962272 30.86719755 32.75082167 44.37712471 39.8716888 53.14247333
 40.91975924 35.87696299]

# 2. Mean, Median, Mode, Variance and Standard Deviation
mean_normal = normal_series.mean()
median_normal = normal_series.median()
mode_normal = normal_series.mode()[0]
variance_normal = normal_series.var()
std_normal = normal_series.std()
print(f"\nMean = {mean_normal:.4f}")
print(f"Median = {median_normal:.4f}")
print(f"Mode = {mode_normal:.4f}")
print(f"Variance = {variance_normal:.4f}")
print(f"Standard Deviation = {std_normal:.4f}")
(23) ✓ 0.0s Python

Mean = 50.1933
Median = 50.2530
Mode = 17.5873
Variance = 95.8864
Standard Deviation = 9.7922

# 3. Statistics of Normal Distribution
minimum_normal = normal_series.min()
maximum_normal = normal_series.max()
print("\nNormal Distribution Statistics:")
print(f"Mean = {mean_normal:.4f}")
print(f"Median = {median_normal:.4f}")
print(f"Variance = {variance_normal:.4f}")
print(f"Standard Deviation = {std_normal:.4f}")
print(f"Minimum = {minimum_normal:.4f}")
print(f"Maximum = {maximum_normal:.4f}")
(24) ✓ 0.0s Python

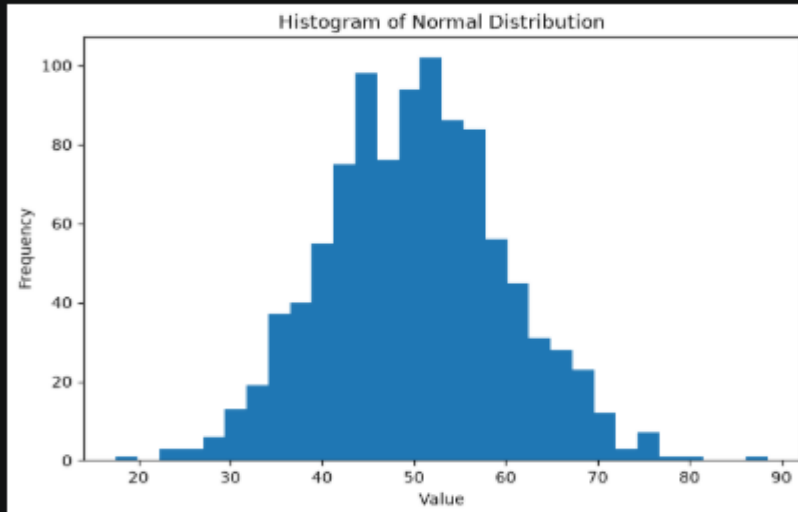
Normal Distribution Statistics:
Mean = 50.1933
Median = 50.2530
Variance = 95.8864
Standard Deviation = 9.7922
Minimum = 17.5873
Maximum = 88.5273

```

```
# 4. Histogram of Normal Distribution
plt.figure(figsize=(8, 5))
plt.hist(normal_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram of Normal Distribution")
plt.show()
```

[25] ✓ 0.0s

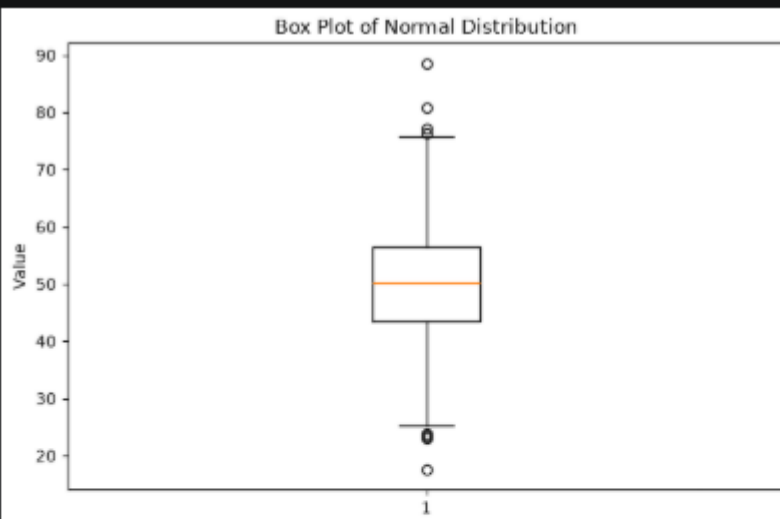
Python



```
# 5. Box plot of Normal Distribution
plt.figure(figsize=(8, 5))
plt.boxplot(normal_data)
plt.ylabel("Value")
plt.title("Box Plot of Normal Distribution")
plt.show()
```

[26] ✓ 0.0s

Python



```
# 6. Percentage of values between 40 and 60
between_40_60 = np.mean(
    (normal_data >= 40) &
    (normal_data <= 60)
) * 100
print(
    f"\nPercentage of values between 40 and 60 = "
    f"{between_40_60:.2f}%"
)
```

[28] ✓ 0.0s

Python

Percentage of values between 40 and 60 = 69.80%

```

# 7. Percentage of values between 30 and 70
between_30_70 = np.mean(
    (normal_data >= 30) &
    (normal_data <= 70)
) * 100
print(
    f"Percentage of values between 30 and 70 = "
    f"{between_30_70:.2f}%"
)
# UNIFORM DISTRIBUTION
uniform_data = np.random.uniform(
    low=0,
    high=100,
    size=1000
)
print("\n----- UNIFORM DISTRIBUTION -----")
print("First 20 Uniform Distribution Values:")
print(uniform_data[:20])
print(f"Mean = {np.mean(uniform_data):.4f}")
print(f"Median = {np.median(uniform_data):.4f}")
print(f"Minimum = {np.min(uniform_data):.4f}")
print(f"Maximum = {np.max(uniform_data):.4f}")

```

(29) ✓ 0.0s Python

Percentage of values between 30 and 70 = 95.98%

----- UNIFORM DISTRIBUTION -----
 First 20 Uniform Distribution Values:
 [16.74825823 10.45678403 63.64382495 70.64757265 3.15861448 93.62122462
 5.19712837 54.12963353 70.90685195 87.09691237 71.40869321 80.17280831
 33.94501925 81.48251137 8.01148464 89.48166561 54.75923762 81.729777
 45.23182845 64.35776952]

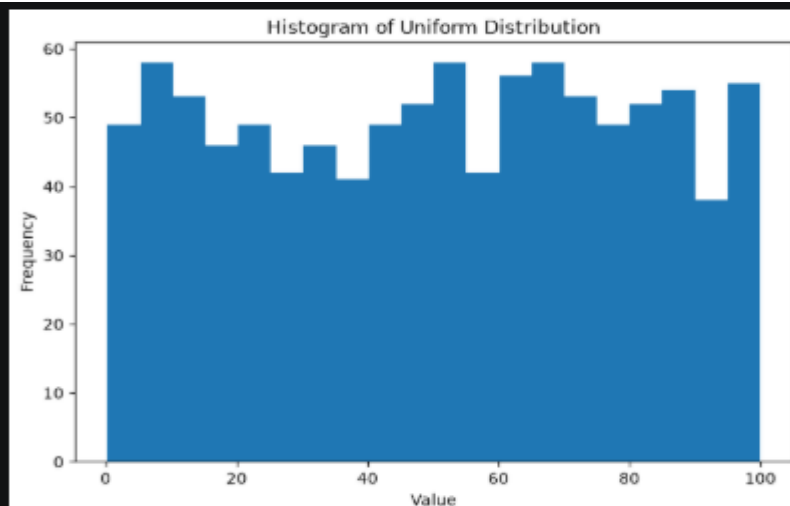
 Mean = 50.3649
 Median = 51.6126
 Minimum = 0.3218
 Maximum = 99.9414

```

# 9. Histogram of Uniform Distribution
plt.figure(figsize=(8, 5))
plt.hist(uniform_data, bins=20)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram of Uniform Distribution")
plt.show()

```

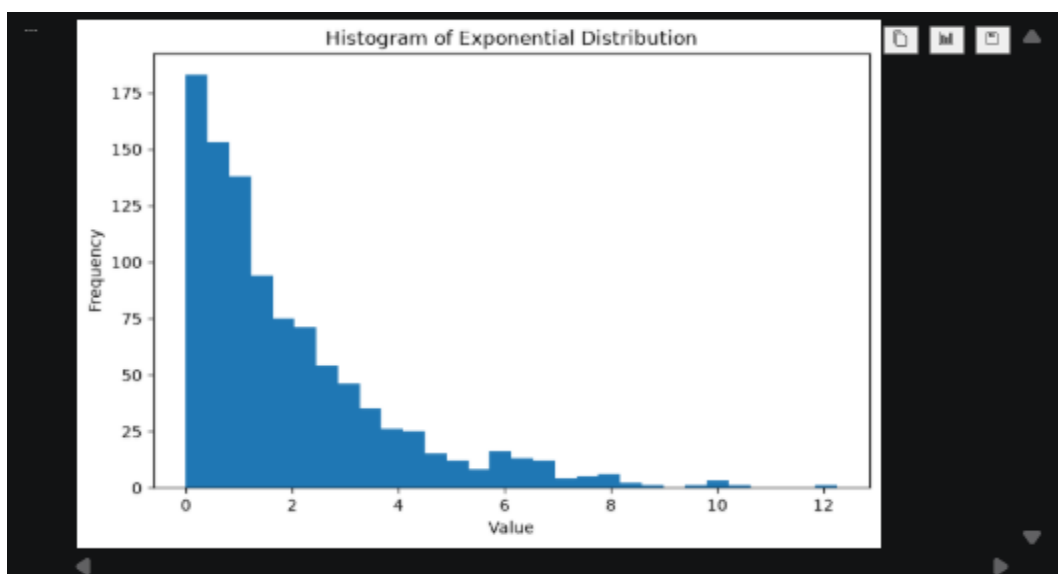
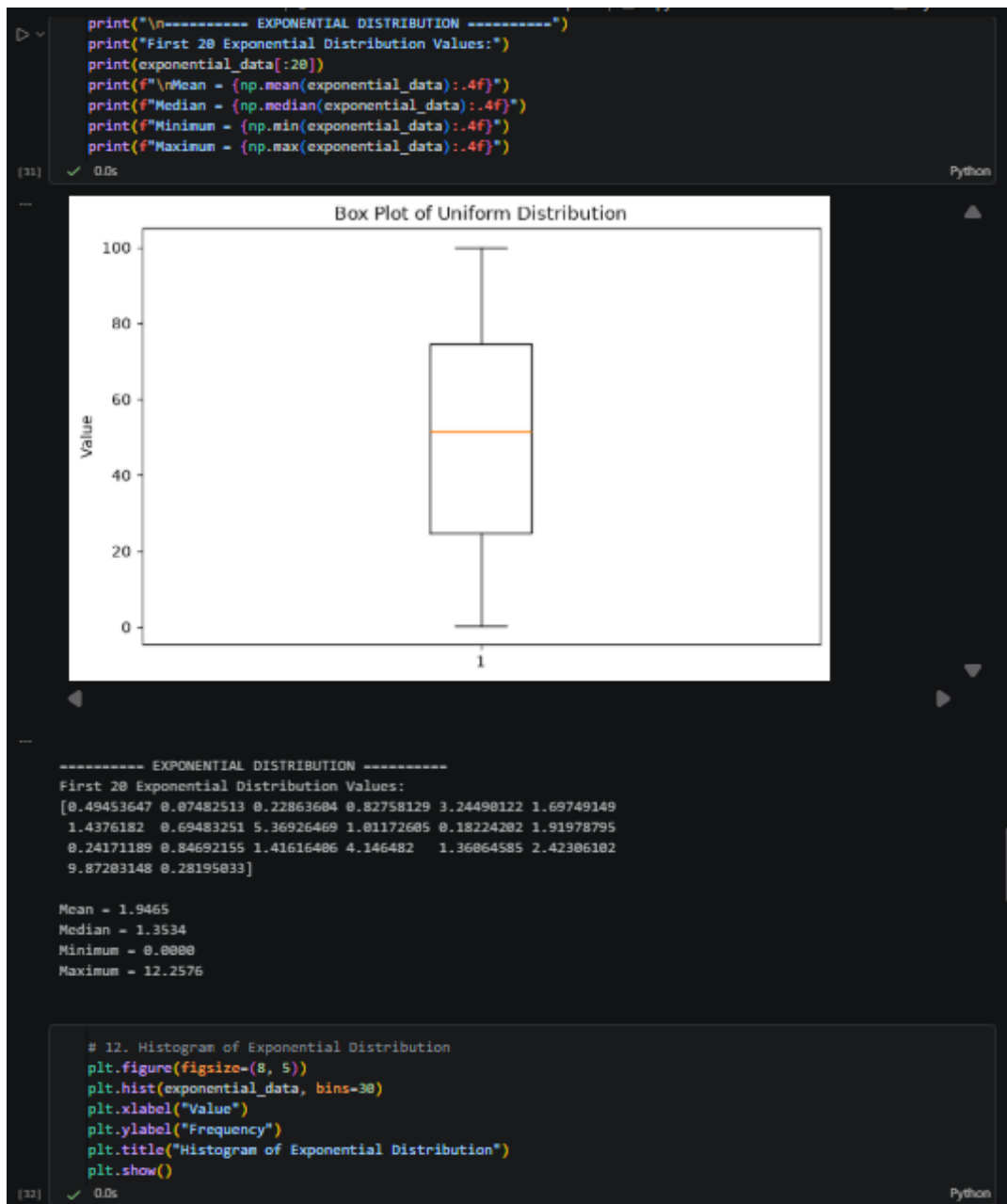
(30) ✓ 0.0s Python

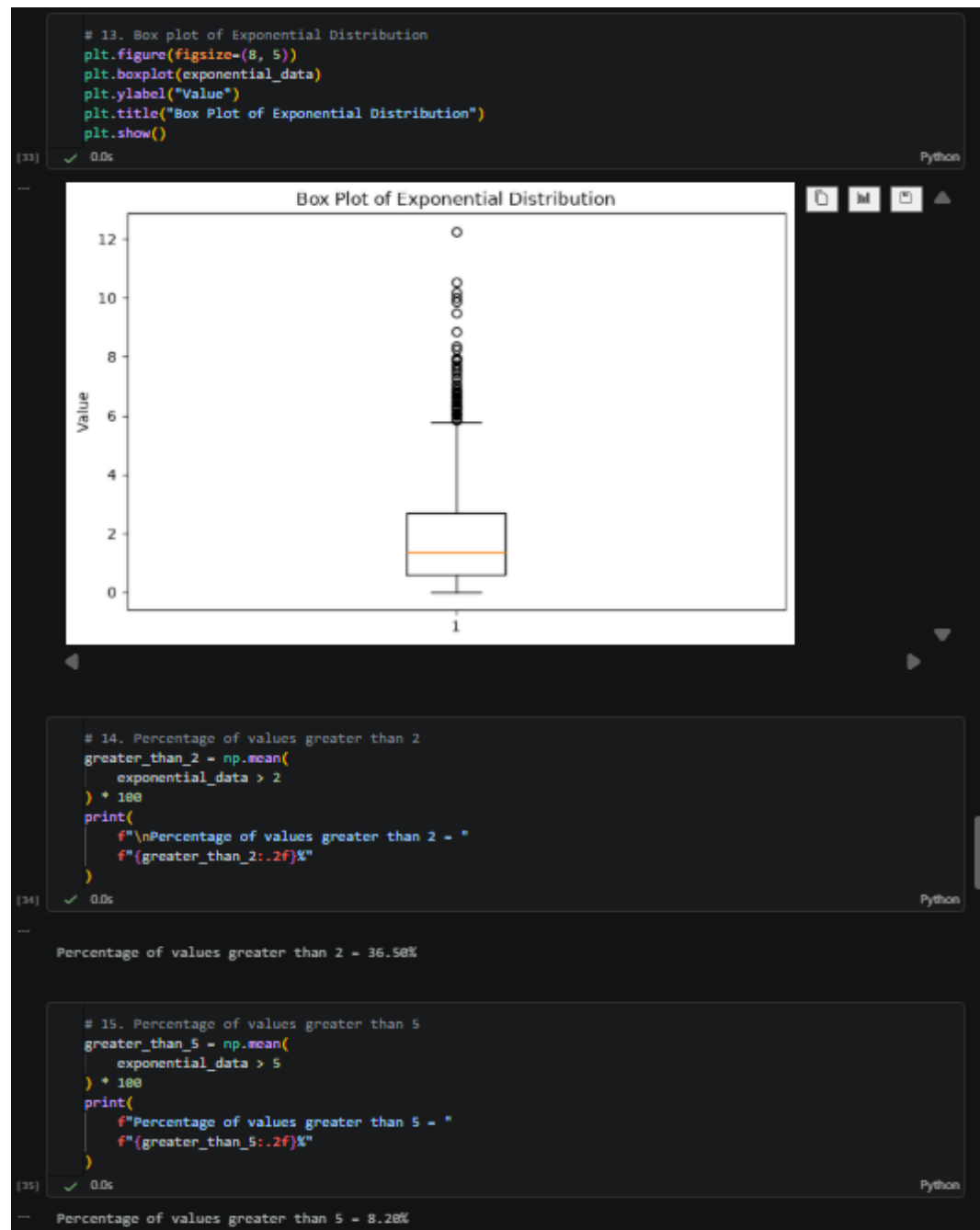


```

# 10. Box plot of Uniform Distribution
plt.figure(figsize=(8, 5))
plt.boxplot(uniform_data)
plt.ylabel("Value")
plt.title("Box Plot of Uniform Distribution")
plt.show()
# EXPONENTIAL DISTRIBUTION
exponential_data = np.random.exponential(
    scale=2,
    size=1000
)

```





C) Coin toss task :

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

print("----- COIN TOSS EXPERIMENT -----")

# 1. Simulate 1,000 coin tosses
tosses = np.random.choice(
    ["Heads", "Tails"],
    size=1000
)
heads = np.sum(tosses == "Heads")
tails = np.sum(tosses == "Tails")
experimental_heads = heads / 1000
experimental_tails = tails / 1000
print("\nTotal Tosses:", 1 (variable) heads: Any
print("Number of Heads:", heads)
print("Number of Tails:", tails)
print("\nExperimental Probability:")
print(f"Heads = {experimental_heads:.4f}")
print(f"Tails = {experimental_tails:.4f}")
```

```

# 3. Bar chart showing frequency of Heads and Tails
plt.figure(figsize=(7, 5))
plt.bar(
    ["Heads", "Tails"],
    [heads, tails]
)
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Heads and Tails")
plt.show()

# 4. Pie chart showing percentage of Heads and Tails
plt.figure(figsize=(7, 7))
plt.pie(
    [heads, tails],
    labels=["Heads", "Tails"],
    autopct="%1.1f%%"
)
plt.title("Percentage Distribution of Heads and Tails")
plt.show()

# 5. Coin tossed 10 times and Heads occurs 8 times
heads_10 = 8
total_10 = 10
probability_heads_10 = heads_10 / total_10
print("\n----- 10 TOSS EXAMPLE -----")
print("Total Tosses = 10")
print("Number of Heads = 8")
print(
    f"Experimental Probability of Heads = "
    f"{probability_heads_10:.2f}"
)

# 6. Simulate 10,000 coin tosses
tosses_10000 = np.random.choice(
    ["Heads", "Tails"],
    size=10000
)
heads_10000 = np.sum(tosses_10000 == "Heads")
tails_10000 = np.sum(tosses_10000 == "Tails")
experimental_heads_10000 = heads_10000 / 10000
experimental_tails_10000 = tails_10000 / 10000
difference_heads = abs(
    experimental_heads_10000 -
    theoretical_heads
)
difference_tails = abs(
    experimental_tails_10000 -
    theoretical_tails
)
print("\n----- 10,000 COIN TOSS ANALYSIS -----")
print(f"Total Tosses = 10000")
print(f"Number of Heads = {heads_10000}")
print(f"Number of Tails = {tails_10000}")
print(
    f"Experimental Probability of Heads = "
    f"{experimental_heads_10000:.4f}"
)
print(
    f"Experimental Probability of Tails = "
    f"{experimental_tails_10000:.4f}"
)
print(
    f"Theoretical Probability of Heads = "
    f"{theoretical_heads:.4f}"
)
print(
    f"Difference between Experimental and Theoretical "
    f"Probability of Heads = {difference_heads:.4f}"
)
print(
    f"Difference between Experimental and Theoretical "
    f"Probability of Tails = {difference_tails:.4f}"
)

# Bar chart for 10,000 tosses
plt.figure(figsize=(7, 5))
plt.bar(
    ["Heads", "Tails"],
    [heads_10000, tails_10000]
)
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("Heads vs Tails - 10,000 Coin Tosses")
plt.show()

# Pie chart for 10,000 tosses
plt.figure(figsize=(7, 7))
plt.pie(
    [heads_10000, tails_10000],
    labels=["Heads", "Tails"],
    autopct="%1.2f%%"
)
plt.title("Percentage Distribution - 10,000 Coin Tosses")
plt.show()
print("\n----- COIN TOSS EXPERIMENT COMPLETED -----")

```

